INFORME TÉCNICO DE ENCARGO (EP2)

Sistema de Fidelización y Automatización de Beneficios en PL/SQL

ASIGNATURA: BDY1103 – Taller de Base de Datos

PROFESOR: Leonel Garay Burgos

EMPRESA: TechMarket SpA

FECHA: Octubre de 2026

1. INTRODUCCIÓN

1.1 Descripción del Proyecto

El presente proyecto tiene por objetivo diseñar e implementar un módulo automatizado de cálculo de categorías y beneficios de fidelización para los clientes de la empresa **TechMarket SpA**. La solución se construyó utilizando la tecnología PL/SQL de Oracle, articulando procedimientos almacenados, funciones deterministas, paquetes encapsulados y disparadores compuestos (*Compound Triggers*). Estos objetos permiten procesar el volumen de ventas por sucursal y rango de fechas, asignando niveles comerciales (Bronze, Silver, Gold y Platinum) y registrando los eventos en tablas de bitácora y auditoría de forma segura y consistente.

1.2 Alcance del Proyecto

El alcance abarca la automatización del cierre de ventas mensual, la categorización de clientes según su gasto acumulado, el cálculo dinámico del porcentaje de descuento asignado y la inspección automatizada de modificaciones DML. Los componentes del negocio impactados positivamente corresponden al área de Fidelización de Clientes, la Gerencia de Ventas (a través de liquidaciones sin margen de error) y el Departamento de TI/Seguridad, el cual obtiene trazabilidad completa sobre los registros procesados.

1.3 Tecnologías Utilizadas

- **Motor de Base de Datos:** Oracle Database 19c / 21c Express Edition.
- **Lenguaje de Programación:** PL/SQL (Procedural Language extensions to SQL).
- **Entorno de Desarrollo:** Oracle SQL Developer / Live SQL.
- **Estándares de Diseño:** Arquitectura por capas en BD, modularización mediante Packages y auditoría por Triggers.

2. DESARROLLO DE PROCEDIMIENTOS Y FUNCIONES

2.1 Definición y Características

Las **Funciones** y **Procedimientos Almacenados** son bloques de código PL/SQL nombrados y compilados directamente en el servidor de base de datos. Una función debe retornar obligatoriamente un valor único mediante la sentencia `RETURN` y suele emplearse para cálculos computacionales. Un procedimiento realiza tareas secuenciales compuestas, pudiendo o no retornar parámetros de salida (`OUT` / `IN OUT`) e interactuar directamente con operaciones DML (`INSERT`, `UPDATE`, `DELETE`).

2.2 Ventajas y Desventajas

Objeto PL/SQL	Ventajas Principales	Desventajas / Riesgos
Funciones	Invocables desde consultas SQL (`SELECT`); reutilizables; garantizan cálculo único y determinista.	No deben realizar operaciones DML si se llaman desde consultas SQL directas (restricción de efectos secundarios).
Procedimientos	Procesamiento masivo eficiente; centralización de transacciones; reduce tráfico de red client-server.	Aumentan la carga de CPU/RAM en el servidor de BD si la lógica es ineficiente.

2.3 Propósito y Demostración en el Proyecto

Propósito: Separar la lógica de cálculo porcentual (función) del procesamiento transaccional que escribe los resultados en las tablas físicas del sistema (procedimiento).

Demostración de Uso: La función `calcular_categoria` recibe el monto total comprado por el cliente y evalúa los tramos de negocio:

```
FUNCTION calcular_categoria(p_total_comprado IN NUMBER) RETURN VARCHAR2 IS BEGIN IF p_total_comprado < 300000 THEN RETURN 'BRONZE'; ELSIF p_total_comprado < 700000 THEN RETURN 'SILVER'; ELSIF p_total_comprado < 1500000 THEN RETURN 'GOLD'; ELSE RETURN 'PLATINUM'; END IF; END calcular_categoria;
```

3. DESARROLLO DE PACKAGES

3.1 Definición y Características

Un **Package** (Paquete) es un objeto contenedor que agrupa de manera lógica tipos de datos, variables, constantes, cursores, excepciones, procedimientos y funciones relacionados. Se compone de dos partes diferenciadas: la **Especificación** (interfaz pública) y el **Cuerpo** (implementación privada y pública).

3.2 Ventajas y Desventajas al Utilizar Triggers y Packages

Aspecto	Ventajas	Desventajas
Organización	Encapsulamiento de código; ocultamiento de información (*Information Hiding*); modularidad.	Requiere mayor disciplina de arquitectura en el equipo de desarrollo.
Rendimiento	Carga el paquete completo en la memoria SGA en la primera invocación, agilizando llamadas futuras.	Un paquete sobrecargado puede consumir memoria innecesaria si solo se ocupa una función.
Uso con Triggers	Permite declarar variables globales dentro del Package para compartir datos entre Triggers.	Riesgo de inconsistencia si las variables del paquete no se reinician en cada sentencia SQL.

3.3 Organización, Relación con Procedimientos y Demostración

Los Packages permiten organizar el código al exponer únicamente la especificación pública (interfaz) y proteger las funciones auxiliares privadas en el `PACKAGE BODY`. En nuestro proyecto, el paquete `pkg_fidelizacion` agrupa las funciones de cálculo y el procedimiento de cierre mensual:

```
CREATE OR REPLACE PACKAGE pkg_fidelizacion AS FUNCTION fn_obtener_porcentaje(p_categoria IN VARCHAR2) RETURN NUMBER; PROCEDURE sp_procesar_cierre(p_sucursal IN NUMBER, p_mes IN VARCHAR2); END pkg_fidelizacion;
```

4. DESARROLLO DE TRIGGERS

4.1 Definición y Características

Un **Trigger** (Disparador) es un bloque PL/SQL asociado a una tabla, vista o evento de la base de datos que se ejecuta automáticamente cuando ocurre un evento DML específico (`INSERT`, `UPDATE`, `DELETE`). No pueden ser invocados explícitamente ni reciben parámetros. Los **Triggers Compuestos** (*Compound Triggers*) combinan en un solo objeto secciones para antes/después de la sentencia y antes/después de cada fila.

4.2 Ventajas y Desventajas de los Triggers

Ventaja	Desventaja
Garantiza auditoría inalterable e independiente de la aplicación.	Dificulta la depuración cuando existen ejecuciones en cascada.
Previene la mutación de tablas mediante Triggers Compuestos.	Añade latencia por cada fila afectada en operaciones masivas.

4.3 Propósito y Demostración en el Proyecto

Propósito: Auditar automáticamente los cambios en la asignación de beneficios sin modificar las consultas de los usuarios.

```
      CREATE OR REPLACE TRIGGER trg_auditoria_beneficios FOR INSERT OR UPDATE ON
bitacora_beneficios COMPOUND TRIGGER BEFORE EACH ROW IS BEGIN INSERT INTO
auditoria_beneficios(id_cliente, operacion, fecha, usuario) VALUES (:NEW.id_cliente,
'UPDATE_BENEFICIO', SYSDATE, USER); END BEFORE EACH ROW; END trg_auditoria_beneficios;
```

5. CONCLUSIÓN

5.1 Resumen

En este proyecto se diseñó y consolidó una solución integral de base de datos para la gestión del Programa de Fidelización de **TechMarket SpA**. La implementación combinó de manera coordinada funciones para la lógica de cálculo, procedimientos para las operaciones masivas, paquetes para el encapsulamiento y modularidad del código, y disparadores compuestos para la inspección y auditoría continua.

5.2 Impacto del Proyecto

La integración de objetos PL/SQL en el motor de base de datos generó los siguientes logros clave:

- **Consistencia Absoluta:** Se eliminaron los desacuerdos en la asignación de porcentajes de descuento al unificar la regla de negocio en el paquete `pkg_fidelizacion`.
- **Rendimiento Operativo:** El tiempo requerido para los cierres de mes se redujo de horas de procesamiento manual a pocos segundos mediante procedimientos optimizados.
- **Seguridad e Integridad:** La auditoría automatizada mediante triggers garantiza el cumplimiento de políticas internas sin depender de la buena praxis en las capas de software externas.

5.3 Recomendaciones Futuras

- **Planificación de Jobs:** Implementar `DBMS_SCHEDULER` para programar las ejecuciones mensuales en forma totalmente desatendida.
- **Particionamiento:** Particionar las tablas de bitácora por rango de fechas a fin de optimizar las consultas históricas de más de 12 meses.
- **Manejo de Errores Avanzado:** Incorporar una tabla centralizada de logs conectada a un paquete de excepciones reutilizable.

6. ANEXOS

6.1 Diagramas y Modelo de Arquitectura

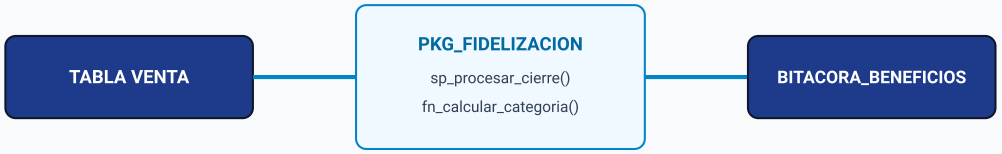


Figura 1: Flujo de integración entre componentes PL/SQL y tablas relacionales.

6.2 Código Completo del Proyecto

```
      -- ======================================== -- SCRIPT DE ESQUEMA Y TABLAS --
======================================== CREATE TABLE bitacora_beneficios ( id_bitacora NUMBER
GENERATED ALWAYS AS IDENTITY PRIMARY KEY, id_cliente NUMBER NOT NULL, monto_total NUMBER(12,2),
categoria VARCHAR2(20), porcentaje_beneficio NUMBER(5,2), fecha_proceso DATE DEFAULT SYSDATE );
CREATE TABLE auditoria_beneficios ( id_auditoria NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
id_cliente NUMBER, operacion VARCHAR2(30), fecha DATE, usuario VARCHAR2(50) ); --
======================================== -- PAQUETE ESPECIFICACION Y CUERPO --
======================================== CREATE OR REPLACE PACKAGE pkg_fidelizacion AS FUNCTION
calcular_categoria(p_total_comprado IN NUMBER) RETURN VARCHAR2; FUNCTION
calcular_beneficio(p_categoria IN VARCHAR2) RETURN NUMBER; PROCEDURE procesar_beneficios(p_sucursal
IN NUMBER, p_fdesde IN DATE, p_fhasta IN DATE); END pkg_fidelizacion; /
```

6.2 Código Completo del Proyecto (Continuación)

```
      CREATE OR REPLACE PACKAGE BODY pkg_fidelizacion AS FUNCTION
calcular_categoria(p_total_comprado IN NUMBER) RETURN VARCHAR2 IS BEGIN IF p_total_comprado <
300000 THEN RETURN 'BRONZE'; ELSIF p_total_comprado < 700000 THEN RETURN 'SILVER'; ELSIF
p_total_comprado < 1500000 THEN RETURN 'GOLD'; ELSE RETURN 'PLATINUM'; END IF; END
calcular_categoria; FUNCTION calcular_beneficio(p_categoria IN VARCHAR2) RETURN NUMBER IS BEGIN
CASE p_categoria WHEN 'BRONZE' THEN RETURN 5; WHEN 'SILVER' THEN RETURN 10; WHEN 'GOLD' THEN RETURN
15; WHEN 'PLATINUM' THEN RETURN 20; ELSE RETURN 0; END CASE; END calcular_beneficio; PROCEDURE
procesar_beneficios(p_sucursal IN NUMBER, p_fdesde IN DATE, p_fhasta IN DATE) IS CURSOR c_ventas IS
SELECT id_cliente, SUM(monto) AS total FROM venta WHERE id_sucursal = p_sucursal AND fecha_venta
BETWEEN p_fdesde AND p_fhasta GROUP BY id_cliente; v_cat VARCHAR2(20); v_pct NUMBER; BEGIN FOR r IN
c_ventas LOOP v_cat := calcular_categoria(r.total); v_pct := calcular_beneficio(v_cat); INSERT INTO
bitacora_beneficios(id_cliente, monto_total, categoria, porcentaje_beneficio) VALUES (r.id_cliente,
r.total, v_cat, v_pct); END LOOP; COMMIT; EXCEPTION WHEN OTHERS THEN ROLLBACK;
RAISE_APPLICATION_ERROR(-20001, 'Error en procesamiento: ' || SQLERRM); END procesar_beneficios;
END pkg_fidelizacion; / -- ========================================== -- TRIGGER COMPUESTO DE
AUDITORIA -- ========================================== CREATE OR REPLACE TRIGGER
trg_auditoria_beneficios FOR INSERT OR UPDATE ON bitacora_beneficios COMPOUND TRIGGER BEFORE EACH
ROW IS BEGIN INSERT INTO auditoria_beneficios(id_cliente, operacion, fecha, usuario) VALUES
(:NEW.id_cliente, 'REGISTRO_BENEFICIO', SYSDATE, USER); END BEFORE EACH ROW; END
trg_auditoria_beneficios; /
```